

Theory Questions

Part 1: Express.js & APIs

Q-1: What is an API?

An **API (Application Programming Interface)** is a software intermediary that enables two applications to communicate by exchanging data and instructions.

Acting as a standardized gateway, it abstracts complex backend logic, allowing frontend applications to securely request and receive data from servers without needing to understand the underlying infrastructure. In the MERN stack architecture, the API serves as the vital link that connects your React interface to your Node.js backend.

Q-2: What is Routing in Express.js?

Routing in Express.js defines how an application responds to client requests at specific endpoints (URLs) using HTTP methods (GET, POST, PUT, DELETE).

Think of it as a traffic controller for your server. When my React frontend requests a specific URL (such as `/api/users`), the Express router intercepts that request and directs it to the corresponding controller or function to send back the required data or perform the necessary action.

Q-3: What is the difference between GET and POST requests?

GET and POST are HTTP methods used by the client to communicate with the server.

GET is for fetching data. It sends parameters via the URL; it is cacheable and used for safe, read-only operations. POST is for submitting data. It sends information in the request body; it is used for state-changing operations like creating or updating resources.

Think of GET as a public query, like searching for a book in a library catalog where the request is visible in the address bar. Think of POST as submitting a private form, where your data is tucked inside an envelope (the request body) to be processed securely by the server.

Part 1: MongoDB & Mongoose

Q-1: What is MongoDB? What is Mongoose?

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents rather than traditional tables. It is designed for scalability and rapid development, making it ideal for handling large, unstructured datasets.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It provides a schema-based solution for modeling your application data, enforcing structure and validation that MongoDB does not natively support.

Q-2: Why do we use Mongoose in a Node.js application?

We use **Mongoose** to bridge the gap between your Node.js application and MongoDB, transforming raw data into structured objects.

Key Benefits:

- **Schema Enforcement:** It defines a strict structure for your data, preventing inconsistent entries.
- **Data Validation:** It automatically validates inputs against your rules (e.g., ensuring an email field is correctly formatted) before saving to the database.
- **Helper Methods:** It provides built-in query builders and middleware, significantly reducing the amount of boilerplate code required for CRUD operations.

Part 1: JWT Authentication

Q-1: What is JWT?

JWT (JSON Web Token) is a secure, compact standard used for stateless authentication.

Think of a JWT as a "digital ID badge". After logging in, the server issues this signed token to the client. The client presents this badge for every subsequent request, allowing the server to verify the user's identity instantly without needing to check a database each time.

Q-2: Why is JWT used in web applications?

JWT is used in web applications to enable stateless authentication and secure data exchange.

Core Advantages:

- **Scalability:** Since the server doesn't need to store session data in a database (stateless), it can easily handle more users and distributed services.
- **Performance:** Verification happens locally using a cryptographic signature, eliminating database lookups for every request.
- **Cross-Domain:** Tokens can be easily shared across different domains or microservices, making it ideal for modern API-driven architectures.

Q-3: What is a Protected Route?

A **Protected Route** is a middleware-based mechanism in web applications that restricts access to specific pages or API endpoints based on the user's authentication or authorization status.

Think of a Protected Route as a "security guard" for my website. When a user tries to access a restricted page (like a dashboard), the route checks for a valid credential (like a JWT). If the user has a valid key, the guard lets them in; if not, the guard redirects them to the login page or displays an "Unauthorized" message.